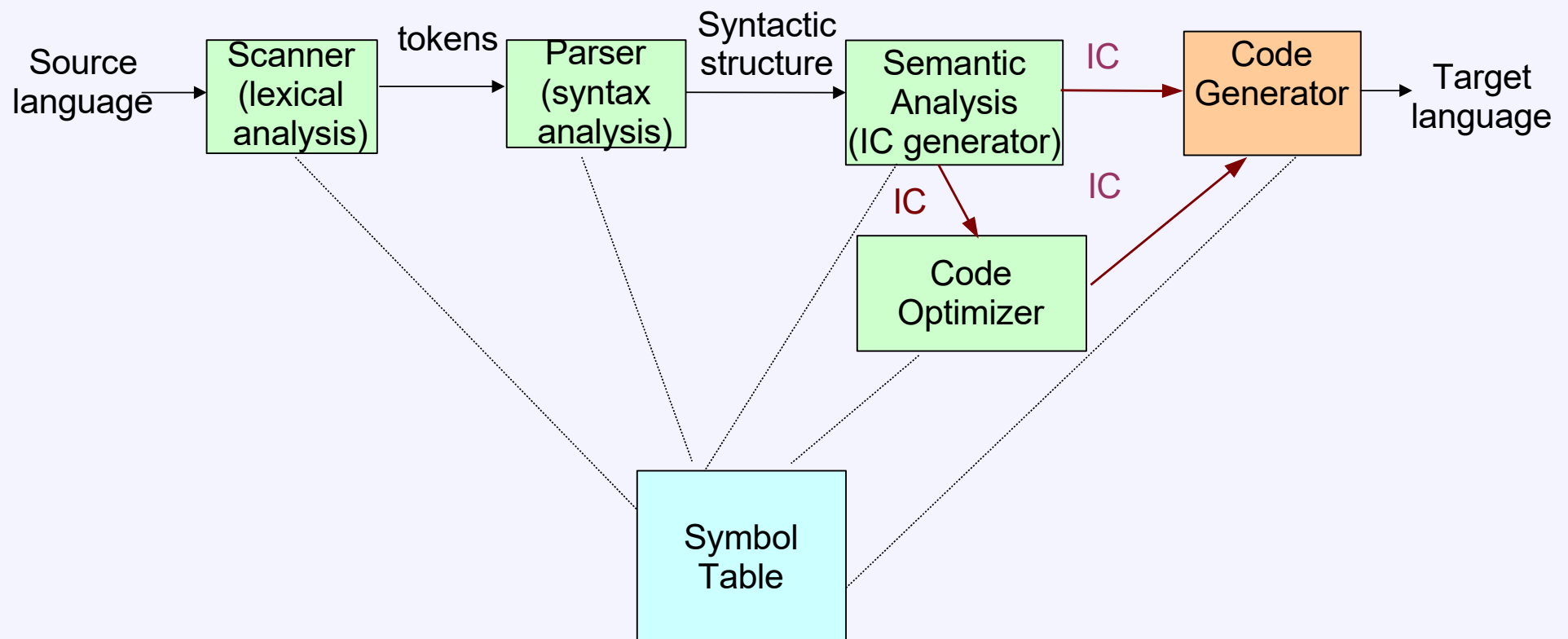


Generovanie kódu a lokálna optimalizácia

Ján Šturc, Richard Ostertág,
Jana Kostíčová

Podľa fialového draka.

Generátor kódu v kompilátore



Cieľové architektúry

- RISC (reduced instruction set computer)
 - trojadresové inštrukcie
 - veľa registrov
 - jednoduchý pamäťový cyklus (málo adresných módov)
- SIC / OISC (single instruction / one instruction set computer)
 - RISC ad absurdum (subleq a,b,c)
($m[b] = m[b] - m[a]$; if ($m[b] \leq 0$) goto c)
- CISC (complex instruction set computer)
 - dvojadresové inštrukcie
 - menej registrov (dnes často procesorová cache)
 - komplikovaný pamäťový cyklus (viac adresných módov)
- SBM (stack based machine)
 - 1961 Burroughs B5000, boli aj ruské pokusy Zebra
 - neosvečilo sa to, mnoho zbytočných swapov
 - myšlienku oživuje JVM (java virtual machine)

Úryvok z nového vydania „dračej knihy“.

In the 1990s, stack-based architectures were revived with the introduction of the Java Virtual Machine (JVM). The JVM is a software interpreter for Java bytecodes, an intermediate language produced by Java compilers. The interpreter provides software compatibility across multiple platforms, a major factor in the success of Java.

To overcome the high performance penalty of interpretation, which can be on the order of a factor of 10, *just-in-time* (JIT) Java compilers have been created. These JIT compilers translate bytecodes during run time to the native hardware instruction set of the target machine. Another approach to improving Java performance is to build a compiler that compiles directly into the machine instructions of the target machine, bypassing the Java bytecodes entirely.

Čo generovať ?

- **Strojový kód**

- Relatívne najjednoduchšie: definujeme si konštanty – symbolické mená inštrukcií.
- Generujeme priamo do pamäti a inštrukcie obsahujú absolútne adresy

- **Binárny relokatívny kód (BRC)**

- Inštrukcie sa generujú rovnako ako v predošlom prípade
- Adresy sú také, že program môže byť umiestnený kdekoľvek v pamäti.
- Loader (linker) to zariadi
- Umožňuje separátnu kompiláciu modulov

- **Assembler**

- Generujeme symbolické mená inštrukcií a symbolické adresy
- Prekladá sa assemblerom
- Zbytočná robota, vyskytuje sa len v študentských projektoch.

Mapovanie medzijazyka do strojového kódu

- Úlohou je mapovať medzijazyk do strojového kódu cieľového stroja. Zložitosť tejto úlohy je určená tromi faktormi
 1. Úroveň medzijazyka
 2. Množina inštrukcií a jej štruktúra (instruction set architecture)
 3. Požadovaná kvalita výsledného kódu
 4. Rýchlosť vykonania inštrukcií a „machine idioms“

Ak nemáme vysoké nároky na efektívnosť výsledného kódu, môžeme každú inštrukciu medzijazyka naprogramovať ako makro alebo šablónu programu a výsledný program skladať z takýchto „makier“.

Ak cieľová architektúra je nehomogénna s veľa obmedzeniami premietne sa to do zložitosti mapovania. Napr. pre operácie s reálnymi číslami sú vyhradené špeciálne registre.

Príklady

$x := y + z;$	LD R_0, y	// $R_0 := y$
	ADD R_0, R_0, z	// $R_0 := R_0 + z$
	ST x, R_0	// $x := R_0$

$a := b + c;$	LD R_0, b	// $R_0 := b$
$d := a + e;$	ADD R_0, R_0, c	// $R_0 := R_0 + c$
	ST a, R_0	// $a := R_0$
	LD R_0, a	// $R_0 := a$
	ADD R_0, R_0, e	// $R_0 := R_0 + e$
	ST d, R_0	// $d := R_0$

$a := a + 1;$	LD R_0, a	// $R_0 := b$
	ADD $R_0, R_0, \#1$	// $R_0 := R_0 + 1$
	ST a, R_0	// $a := R_0$

Machine idiom:
INC $a, \#1$

Model cieľového počítača

Presuny:

LD $\langle r_i \rangle$, $\langle \text{address} \rangle$

ST $\langle r_i \rangle$, $\langle \text{address} \rangle$

MV r_i , r_j

Prečo nie univerzálna inštrukcia:

MV $\langle \text{destination} \rangle$ $\langle \text{source} \rangle$?

Výpočtové operácie:

OP $\langle r_i \rangle$, $\langle r_j \rangle$, $\langle r_k \rangle$ Skoky: // $r_i := r_j \text{ op } r_k$

JMP $\langle \text{address} \rangle$ BR $\langle \text{cond} \rangle$, $\langle r_i \rangle$,

Spôsoby adresácie $\langle \text{address} \rangle$

r = hodnota v registri r

- **#a** operand je konštanta
- **a bez modifikácie** $\langle \text{address} \rangle = a$, operand = $*(a)$
- ***a** nepriama adresa $\langle \text{address} \rangle = *(a)$, operand = $*(*(a))$
- **a(r)** modifikácia registrom $\langle \text{address} \rangle = a + r$, operand = $*(a + r)$ - polia
- **100(r)** relatívne adresy $\langle \text{address} \rangle = r + 100$, operand = $*(r + 100)$ - base+offset

Prehľad spôsobov adresácie

MODE	FORM	ADDRESS
absolute	M	M
register	R	R
indexed	c(R)	c + contents(R)
indirect register	*R	contents(R)
indirect indexed	*c(R)	contents(c+contents(R))
literal	#c	c
stack	SP	SP
indexed stack	c(SP)	c + contents(SP)

Dĺžka a cena inštrukcií

1. Počet inštrukcií	16 – 981
2. Počet registrov	4 – 64
3. Priamo adresovateľná pamäť	256 – 16G
4. Počet spôsobov adresácie	1 – 8

3 typy inštrukcií

1. typ: OP r MOD a	48 bit
2. typ: OP r_i, r_j	16 bit
3. typ: OP r_i, r_j, r_k	24 – 32 bit

Významnú úlohu hrajú aj:

- šírka dátovej zbernice
- šírka inštrukčnej zbernice

Cena inštrukcie = počet pamäťových cyklov

Pamäťový cyklus: prečítanie a obnovenie.

Adresný cyklus: výpočet adresy (obvykle jeden pamäťový cyklus za každú indirekciu).

Inštrukčný cyklus:

- Načítanie inštrukcie, dekodovanie inštrukcie
- Výpočet adresy
- Načítanie operandu
- Vykonanie operácie

Špecializované a externé zariadenia

- Prúdové spracovanie (SIMD, pole procesorov, pipe)
 - Napr. násobičky, sčítačky
 - Predspracovanie (potrebná administrácia), inicializácia
 - Oneskorenie (prvý výsledok)
 - Perióda čas medzi dvomi po sebe idúcimi výsledkami
- Cache (často 2 – 3 úrovne)
 - inštrukčná
 - dátová
- Externé DMA blokové prenosy
 - Štartovacia inštrukcia, potom už pracuje paralelne
 - Dvojbufrová administrácia
 - RAID

Podľa „hardvéristov“ sú tieto veci pre používateľa transparentné.
Skutočnosť: program generovaný kompilátorom, ktorý pozorne nezohľadňuje možnosti týchto zariadení, dosahuje len zlomok možného výkonu.

Pipeline – prúdové spracovanie

IF	ID	MA	EX	WB		
	IF	ID	MA	EX	WB	
		IF	ID	MA	EX	WB

IF – Instruction Fetch
ID – Instruction Decode
MA – Memory access
EX – Execute
WB – Write back

Načítanie inštrukcie
Dekódovanie inštrukcie
Načítanie operandu
Vykonanie inštrukcie
Obnovenie pamäti

Konflikty – hazards

Štrukturálne:
Ohraničenia na súčasné použitie zdrojov počítača.

Dátové:
Potrebujeme výsledok ešte neskončenej inštrukcie.

Riadenia:
Podmienené skoky

Riešenie konfliktov:

Dátové a štrukturálne konflikty sa dajú riešiť pozdržaním (zariadením prázdneho taktu).

Konflikty spôsobené skokmi vyžadujú zrušenie časti výpočtu a začať znovu. Rafinované stratégie a odhady.

Prideľovanie registrov

- Dve úlohy
 - **Register allocation:** Pre každý bod programu určíme množinu premenných, ktorá bude v danom bode v registroch
 - **Register assignment:** Vyberáme konkrétny register pre danú premennú.
- Register assignment je matematicky (NP) ťažká úloha
 - Ďalšou komplikáciou je skutočnosť, že niektoré operácie vyžadujú konkrétne dvojice registrov. (napr. MQ pre IBM370).
 - Párny + nepárny register pre operácie s dvojitou dĺžkou slova.

Príklad: $t := a + b$ LD R₀, #0
 $t := t * c$ LD R₁, a
 $t := t / d$ ADD R₁, b
MUL R₀, c
DIV R₀, d
ST R₁, t

Pracuje s dvojicou R₀:R₁

$t := a + b$ LD R₀, a
 $t := t + c$ ADD R₀, b
 $t := t / d$ ADD R₀, c
SRDA R₀, #32
DIV R₀, d
ST R₁, t

Shift Right
Double
Arithmetic

Správa pamäti počas behu

- Pre každú procedúru tri oblasti (areas)
 - Oblasť kódu (do tej sa generuje kód, je statická)
 - Oblasť statických dát
 - Oblasť dát na zásobníku
- Halda je spoločná a riadi sa explicitnými príkazmi alokácie a dealokácie.
- Volajúci musí:
 - Alokovat' na zásobníku priestor pre volací záznam volaného.
 - Uložit' doň potrebné údaje.
 - Vyhradiť priestor pre vrátené hodnoty.
 - Odovzdať riadenie volanému (skočiť na jeho vstupnú adresu v oblasti kódu).
- Volaný musí
 - Vykonať prácu
 - Upratať po sebe na zásobníku
 - Vrátiť riadenie volanému

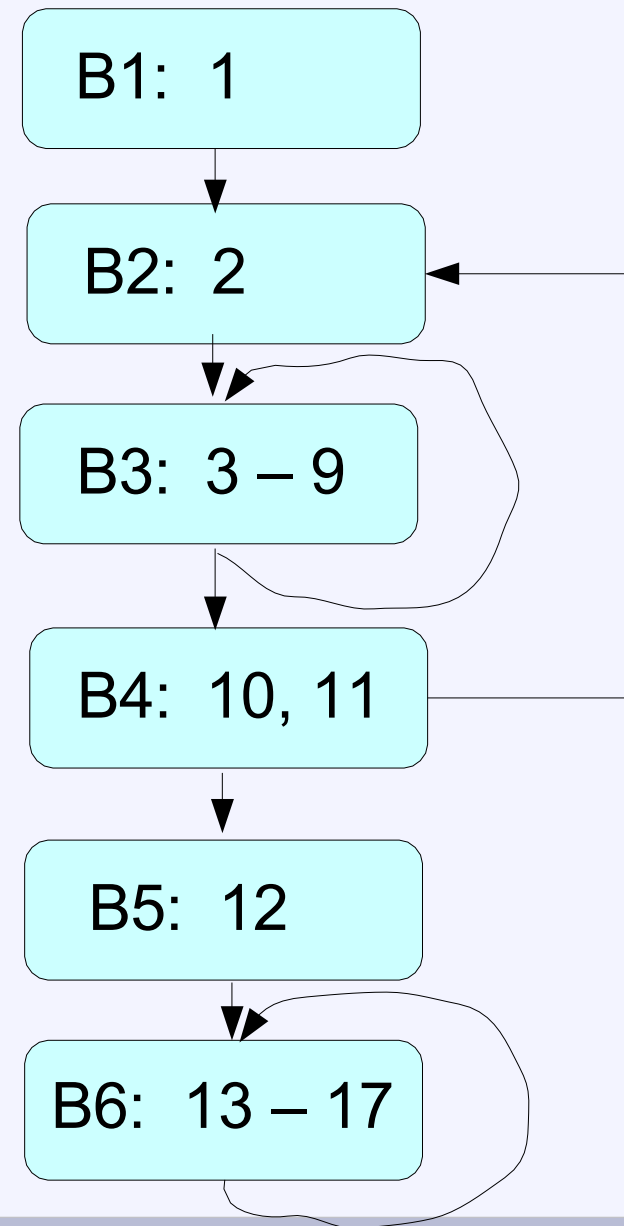
Rozdelenie programu na bloky

- Blok (interval) je postupnosť za sebou idúcich príkazov programu, ktorá, keď sa začne vykonávať, sa vždy vykoná celá.
- Blok sa skladá z hlavičky (leader) a ostatných príkazov bloku.
- Algoritmus rozdelenia na bloky spočíva v určení hlavičiek.
 - Prvý príkaz programu je hlavička bloku.
 - Každý cieľ skoku je hlavička bloku.
 - Príkaz bezprostredne nasledujúci za príkazom skoku je hlavička bloku.
 - Neexistujú iné hlavičky blokov.
- Blok je úsek programu začínajúci hlavičkou po nasledujúcu hlavičku.
- Blok obsahuje najviac jeden skok (na svoju hlavičku).

Príklad

```
for (i=1, i <= 10, i++)  
  for (j=1, i <= 10, j++)  
    a[i, j] = 0.0;  
for (i=1, i <= 10, i++)  
  a[i, i] = 1.0;
```

```
1. i:= 1  
2. j:= 1  
3. t1:= 10*i  
4. t2:= t1 + j  
5. t3:= 8*t2  
6. t4:= -88 + t3  
7. a[t4]:= 0.0  
8. j:= j + 1  
9. if j ≤ 10 goto 3  
10. i:= i + 1  
11. if i ≤ 10 goto 2  
12. i:= 1  
13. t5:= i - 1  
14. t6:= 88*t5  
15. a[t6]:= 1.0  
16. i:= i + 1  
17. if i ≤ 10 goto 13
```



Next use information (ďalšie použitie)

- Definícia p. $x := \dots$ má ďalšie použitie, ak existuje príkaz s, používajúci x a cesta v grafe toku riadenia z p do s taká, že hodnota x sa počas nej nezmení.
Hovoríme, že príkaz s používa hodnotu x vypočítanú príkazom p.
- Premenná, ktorá má ďalšie použitie je živá.
- Premenná, ktorá nie je živá, je mŕtva.
- „Živost“ je o tom, či sa musíme ešte zaujímať o hodnotu danej premennej.

Algoritmus pre výpočet next use v bloku

Vstup: základný blok B, tabuľka symbolov

Výstup: pre každý priradovací príkaz i. $x := \text{op}(x_1, \dots, x_k)$
v bloku B informácia o živosti premenných x a next use i.

Metóda:

- Všetky nedočasné premenné (nontemporary) na výstupe z bloku B sú už v tabuľke symbolov označené ako živé.
- for(i = last statement of B, i >= first statement of B, i--)
{ Ak i je tvaru $x := \text{op}(x_1, \dots, x_k)$:
 1. Pre i nastav „next use“ podľa „next use“ x z tab. sym.;
 2. V tabuľke symbolov ulož „next use“ a „live“ x = false;
 3. V tabuľke sym. ulož „live“ $x_1, \dots, x_k = \text{true}$ a „next use“ pre ne je príkaz i;}

RISC vs CISC – pripomenutie

	RISC	CISC
Zložitosť inštrukcií	Jednoduchšie	Zložitejšie
Prístup do pamäte	Iba load/store inštrukcie; ostatné inštrukcie pracujú s registrami	Aj niektoré inštrukcie pre operácie, napr. ADD (často druhý operand môže byť v pamäti)
Dĺžka inštrukcie	Fixná (väčšinou 32 bitov)	Variabilná
Adresné módy	Jednoduchšie	Zložitejšie
Počet registrov	Viac	Menej
Pipelining	Jednoduchší	Zložitejší
Formát inštrukcie	Väčšinou trojadresné	Väčšinou dvojadresné
Veľkosť programu	Zaberá viac miesta	Zaberá menej miesta
Programovanie v ASM	Náročnejšie – komplexita na strane softvéru	Jednoduchšie – komplexita na strane hardvéru
Príklady	ARM (proprietárne), RISC-V (open source)	x86 / x86-64 (proprietárne)

Rozsah optimalizácie

Lokálna (základný blok)

- Je to najjednoduchšie
- Dá sa robiť súčasne s generovaním kódu.

Globálna (intraprocedurálna)

- Optimalizujú sa jednotlivé procedúry programu
- Viac ako blok, ale ešte zvládnuteľné
- Dôležité lebo cykly väčšinou presahujú jeden základný blok

Interprocedurálna (celý program, modul)

- Je to „skoro to isté“ ako globálna len vo väčšom rozsahu.
- Umožňuje niektoré nové optimalizácie (napr. inlining, špecializáciu volania ak procedúra nie je rekurzívna, ...)
- Vyňatie volania procedúry pred cyklus.
- Veľa (najmä historických) kompilátorov nerobilo interprocedurálnu optimalizáciu. Moderné kompilátory – často voliteľná možnosť.

Lokálne optimalizácie pomocou DAGu

Dag základného bloku

- Pravidlá vytvorenia dagu:
 1. Pre každú pôvodnú hodnotu premennej v bloku vytvoríme uzol - list.
 2. Pre každý príkaz s bloku B vytvoríme uzol N . Synovia uzlu N sú vrcholy zodpovedajúce **poslednej definícii operandov N** predchádzajúcej príkaz s.
 3. Uzol N označíme operáciou príkazu s a zoznamom premenných pre ktoré je poslednou definíciou v bloku.
 4. Niektoré uzly označíme ako výstupné uzly (sú to tie uzly, ktorých premenné sú živé po výstupe z bloku). Výpočet živých premenných na výstupe je predmetom **globálnej analýzy toku dát**.

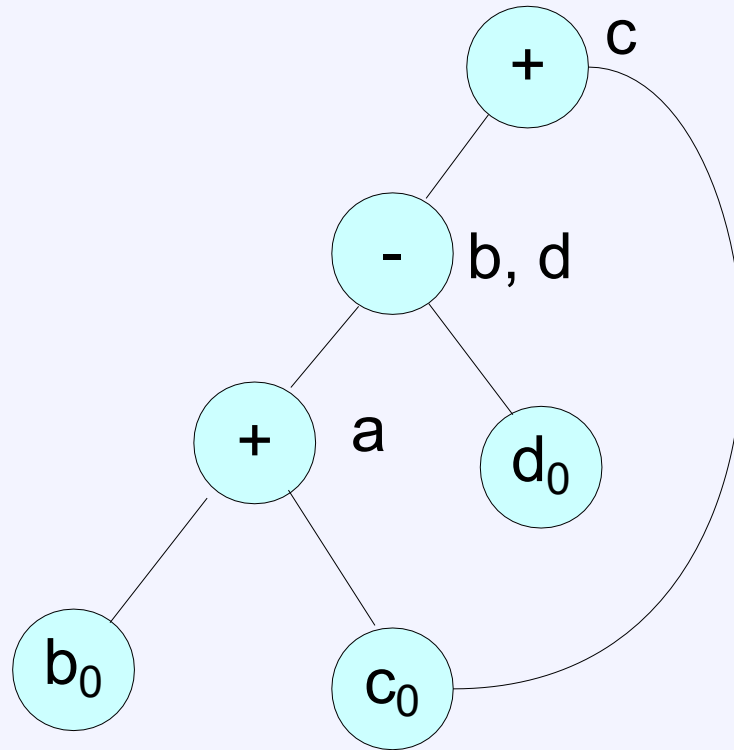
- Použitie dagu

- Eliminácia lokálnych spoločných podvýrazov
- Eliminácia mŕtveho kódu
- Preusporiadanie nezávislých príkazov
- Využitie algebraických zákonov – preusporiadanie operandov

Premenné sú
predtým odstránene z
ostatných uzlov
(„killed“)

Príklad – eliminácia spoločných podvýrazov

$a := b + c$
 $b := a - d$
 $c := b + c$
 $d := a - d$



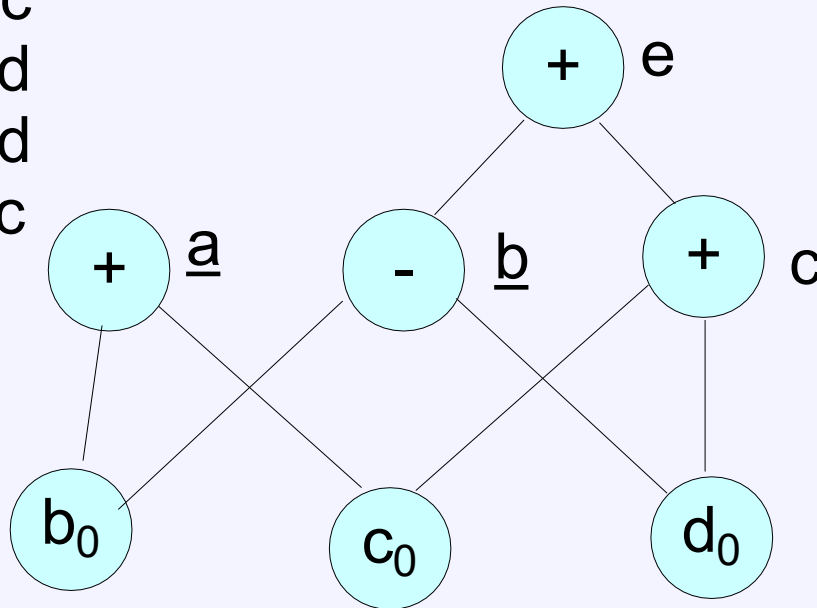
$a := b + c$
 $b := a - d$
 $c := b + c$
 $d := b$

Až globálna analýza toku dát zistí, či premenná d je naozaj zbytočná.

Postup ako pri tvorbe DAGu v prednáške o preklade do medzijazyka

Príklad – eliminácia mŕtveho kódu

$a := b + c$
 $b := b - d$
 $c := c + d$
 $e := b + c$



Pozn.: $e = a$

Posledné priradenie je zbytočné, lebo

$b + c = (b - d) + (c + d)$.

Tento fakt eliminácia spoločných podvýrazov neodhalí.

Využitie algebraických zákonov ?

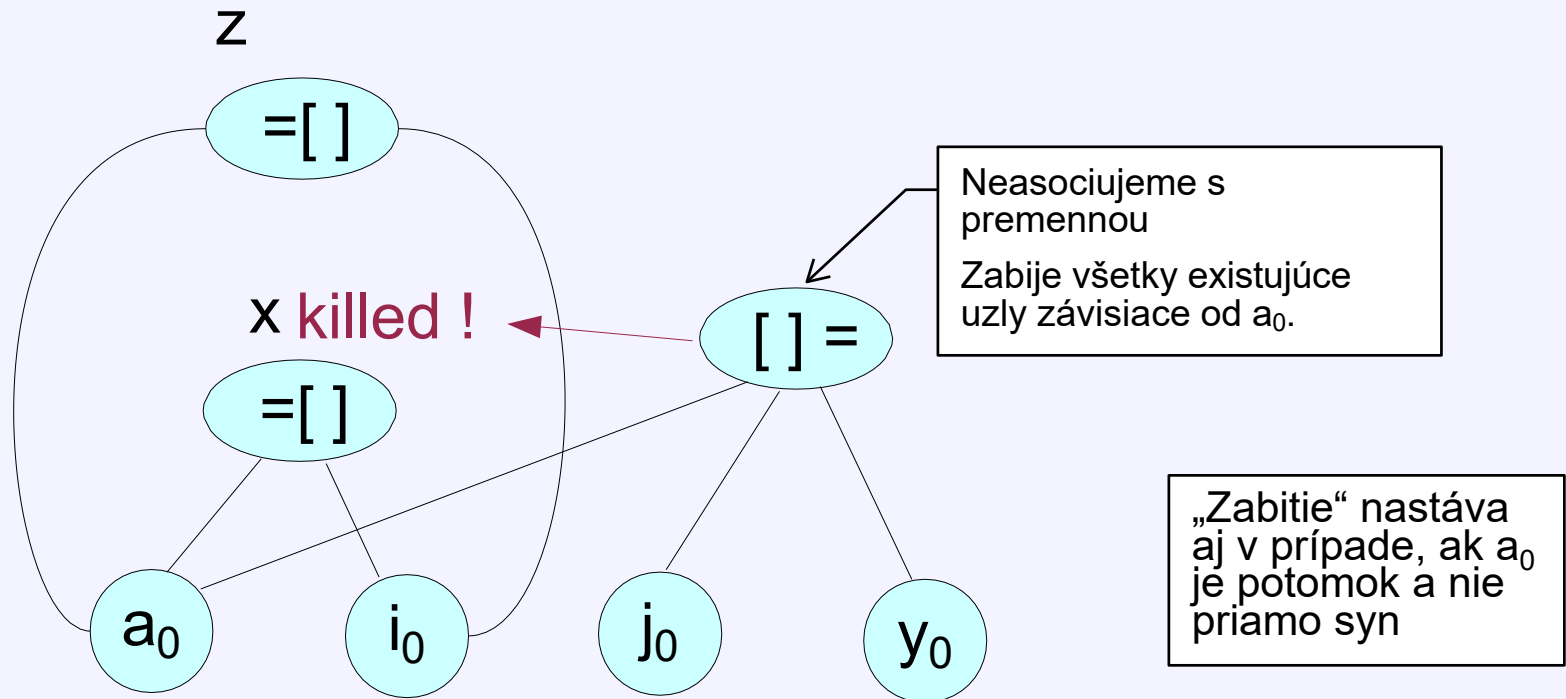
Ak a , b sú živé a c , e nemajú následné použitie, je ich výpočet zbytočný. Po eliminácii mŕtveho kódu:

$a := b + c$
 $b := b - d$

Odstraňujeme postupne korene, ktoré nemajú asociované živé premenné

Polia vyžadujú opatrnosť

$x := a[i]$
 $a[j] := y$
 $z := a[i]$



Nemôžeme využiť x pre výpočet z , lebo nevieme, či náhodou nie je $i = j$.

Podobne: **smerníky, volania procedúr**

- $x = *p$ – použitie všetkých premenných, všetky uzly s identifikátormi sú argumenty
- $*q = y$ – zápis do všetkých premenných, „zabije“ všetky pred týmto operátorom urobené priradenia
- **Procedúra** – môže použiť / zmeniť dáta, ku ktorým má prístup

Poradie výpočtu

- Poradie výpočtu musí rešpektovať usporiadanie v dagu. Vyhodnotenie synov musí predchádzať vyhodnotenie otca.
- Polia, smerníky a volania procedúr musia byť vyhodnotené v takom poradí, ako v pôvodnom neoptimalizovanom bloku.
 - Priradenie do poľa musí nasledovať za všetkými predchádzajúcimi* priradeniami do tohto poľa / použitiu tohto poľa
 - Použitie prvkov poľa musí nasledovať za všetkými predchádzajúcimi* priradeniami do tohto poľa
 - Použitie premennej musí nasledovať za všetkými predchádzajúcimi* volaniami procedúr alebo nepriamymi priradeniami cez smerník
 - Volanie procedúry alebo nepriame priradenie cez smerník musí nasledovať za všetkými predchádzajúcimi* použitiami akejkolvek premennej

Algebraické zákony

- Identity

- $x + 0 = 0 + x = x;$ $x - 0 = x$
- $1 * x = x * 1 = x;$ $x / 1 = x$

- Redukcia v sile

- $x^2 = x * x$
- $2 * x = x + x = x \ll 1 \ x / 2$
- $= 0.5 * x$
-

uint16_t x:

- $x / 10 = ((\text{uint32_t})x * (\text{uint32_t})0xCCCCD) \gg 19)$
- $x / \pi = (((\text{uint32_t})x * (\text{uint32_t})0x45F3) \gg 16) + x) \gg 2)$

- Skladanie konštánt (constant folding) počas kompilácie.

int x = 2 + 3 * 4; -> int x = 14;

- Komutatívne zákony

- Zdanlivo nič neoptimalizujú, ale keď jeden operand je už v registri môžeme inštrukciu LD ušetriť.

Alokácia a pridelovanie registrov

Význam registrov

- Pre väčšinu architektúr aspoň jeden operand (niekedy aj všetky operandy) musia byť v registroch, aby sa operácia dala vykonať.
- Registre sú vhodné miesto pre uchovanie medzivýsledkov a dočasných premenných v rámci bloku.
- Registre sú vhodné miesto pre niektoré globálne premenné (premenná cyklu, indexy do polí) používané vo viacerých blokoch.
- Registre sa používajú na údržbu dátových štruktúr pre podporu počas behu. Napr. smerník na vrchol zásobníka a rôzne pomocné smerníky.

Dátové štruktúry pre prácu s registrami

- Register descriptor (čo je v reg.)
 - Pre každý (dostupný) register eviduje premennú, ktorej hodnota je v registri.
 - Môžeme predpokladať (pokiaľ evidujeme len registre pre daný základný blok), že na počiatku je register descriptor prázdny.
- Address descriptor (kde všade je premenná)
 - Pre každú premennú programu udržiava miesto (miesta), kde sa okamžitá hodnota danej premennej nachádza.
 - Miesto môže byť register, adresa v pamäti, miesto v zásobníku, v halde a pod.
 - Vhodné miesto pre udržiavanie address deskriptoru je príslušná položka tabuľky symbolov.
- Proces generovania kódu je sprevádzaný údržbou týchto štruktúr.

Jednoduchý algoritmus generovania kódu

- Predpokladáme, že máme postupnosť inštrukcií v medzijazyku (pregenerovaných pomocou dagu).

```
for (i=leaderB; i < nextleader; i++)  
    { LR = GetReg(instruction[i]); // vráti registre Rx, Ry, Rz  
      CodeGen(instruction[i], LR); // vygeneruje strojový kód  
      Update(register_descriptor);  
      Update(address_descriptor)      // instruction[i]: x = y op z  
    }
```

- Procedúry GetReg a CodeGen využívajú informácie z register descriptoru a address descriptoru. Sú navrhnuté tak, aby negenerovali zbytočné presuny medzi registrami.

CodeGen generuje inštrukciu LD R_y,y' ak y nie je v R_y (y' je adresa y v pamäti). Podobne pre z a R_z.

Príklad

```
t := a - b
u := a - c
v := t + u
a := d
d := v + u
```

t, u, v sú dočasné premenné

a, b, c, d existujú aj po skončení bloku

Po copy inštrukcii môžeme mať viac premenných v jednom registri

3-ADDRESS INSTRUCTION	MACHINE CODE	REGISTER DESCRIPTOR	ADDRESS DESCRIPTOR
t = a - b	LD R1, a LD R2, b SUB R2, R1, R2	R1 has {a} R2 has {b} R2 has {t}	a in {a, R1} b in {b, R2} b in {b} t in {R2}
u = a - c	LD R3, c SUB R1, R1, R3	R3 has {c} R1 has {u}	c in {c, R3} a in {a} u in {R1}
v = t + u	ADD R3, R2, R1	R3 has {v}	c in {c} v in {R3}
a = d	LD R2, d	R2 has {d} R2 has {a, d}	d in {d, R2} a in {R2}
d = v + u	ADD R1, R3, R1	R1 has {d}	d in {R1}
end	ST a, R2 ST d, R1		a in {a, R2} d in {d, R1}

Stratégia pre GetReg($x := y \text{ op } z$) 1

- Najprv potrebujeme registre pre operandy y, z
 1. Ak y je už v registri vráť tento register ako R_y . Zistíme z address descriptoru
 2. Ak y nie je v registri, ale existuje register, ktorý obsahuje mŕtvu premennú vráť tento register.
 3. Ani (1) ani (2), ale je k dispozícii prázdny register, tak vráť tento register.
 4. Ak ani prázdny register nie je k dispozícii, tak ak x je v registri a x nie je druhý operand vráť tento register. Aj tak sa prepíše
 5. Nič z predošlých bodov, nájdi vhodného kandidáta: register R . Register R obsahuje premennú v .
 - Ak address descriptor pre v hovorí, že premenná v je uložená aj inde, môžeš vrátiť register R .
 - R je jediné miesto pre premennú v potom „spill“ generuj inštrukciu **ST R v**, uprav address descriptor a vráť register R .
- Stratégia pre druhý operand je rovnaká.

Stratégia pre GetReg($x := y \text{ op } z$) 2

- Register pre výsledok R_x
 1. Pretože sa počíta nová hodnota x register obsahujúci iba x je najvhodnejší.
 2. Ak neexistuje a aspoň jeden z operandov je mŕtvy (nemá v bloku next use) alebo je uložený aj inde je register obsahujúci iba operand vhodný.
 3. Ďalej môžeme skúsiť inú mŕtvu premennú a prázdny register
 4. Pokračujeme ako pri hľadaní registrov pre operandy.
 - Stratégia pre GetReg($x := y$)
 - Nájdeme register R_y pre y ako v predošlom prípade.
 - Register pre výsledok bude vždy ten istý $R_x = R_y$.
- CodeGen nemusí generovať $x := y$, stačí ak poznačí, že register R_y obsahuje aj x .

Globalizácia problému registrov

- Pri lokálnom prideľovaní registrov sa na konci bloku premenné zapamätajú do pamäti.
- Mnohé premenné a cykly prechádzajú cez viac blokov.
- Predpokladáme, že ak premenná x je počas cyklu L
- v registri, pri každom jej použití ušetríme **jeden pamäťový cyklus**.
- Celkový zisk pre premennú x v registri je daný vzorcom:

Počet prípadov, keď x je použité v B pred prvým priradením do x v B .

+2 za ušetrenie ST inštrukcie

1 ak x je živé na konci B a bolo doň v B priradené
0 inak (nepotrebovali sme ukladať)

$$\sum_{B \in L} (use(x, B) + 2 * live(x, B))$$

+1 za prístup do registra namiesto do pamäte

- Počíta sa cez všetky bloky v rámci cyklu
- Je to približné vyjadrenie - nie všetky bloky sa vykonávajú rovnaký počet krát.

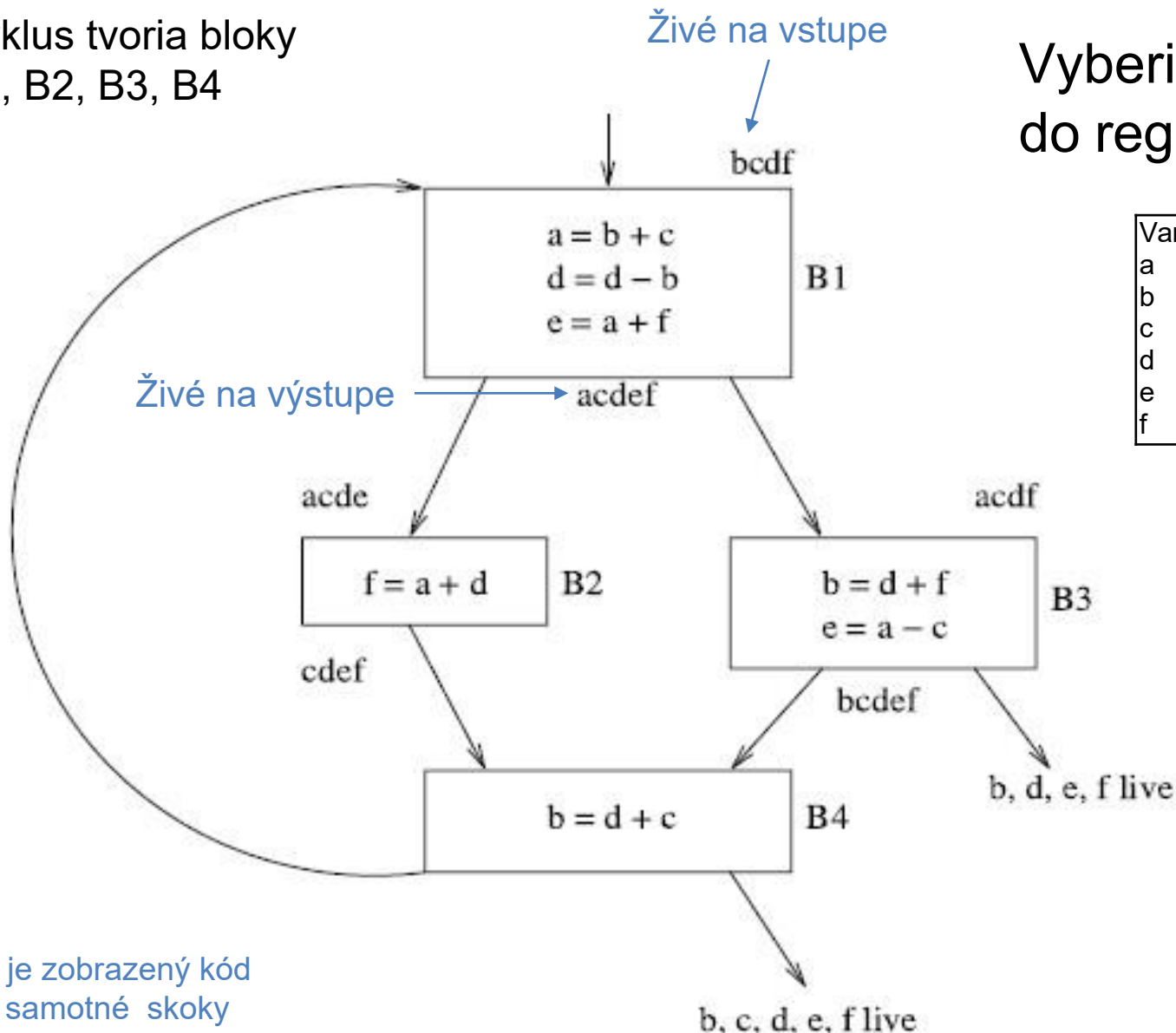
Príklad – medzijazyk

Cyklus tvoria bloky
B1, B2, B3, B4

Živé na vstupe

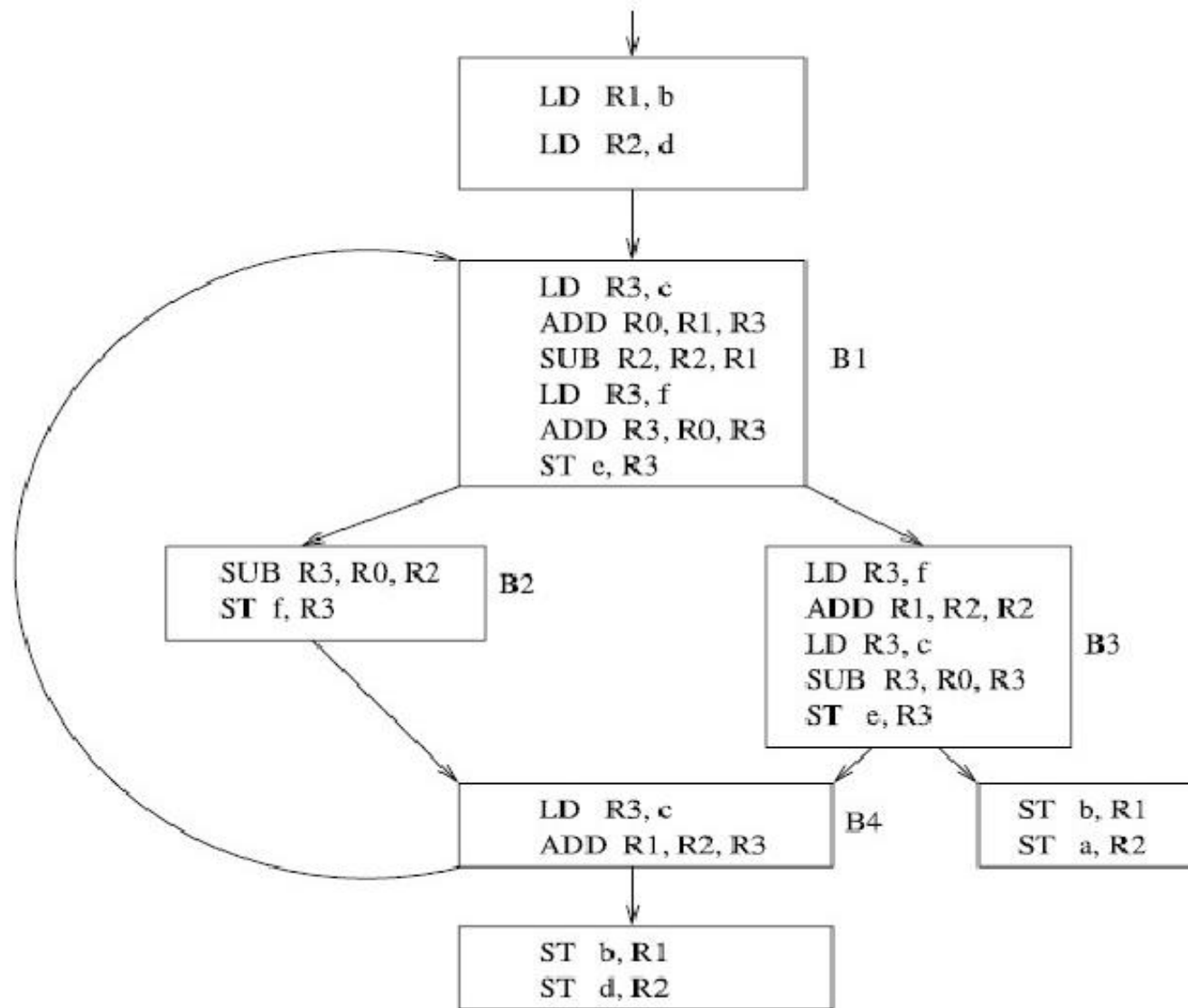
Vyberieme a, b, d
do registrov:

Var	Hod	nota	Σ
a	4		R0
b	6		R1
c	3		
d	6		R2
e	4		
f	4		



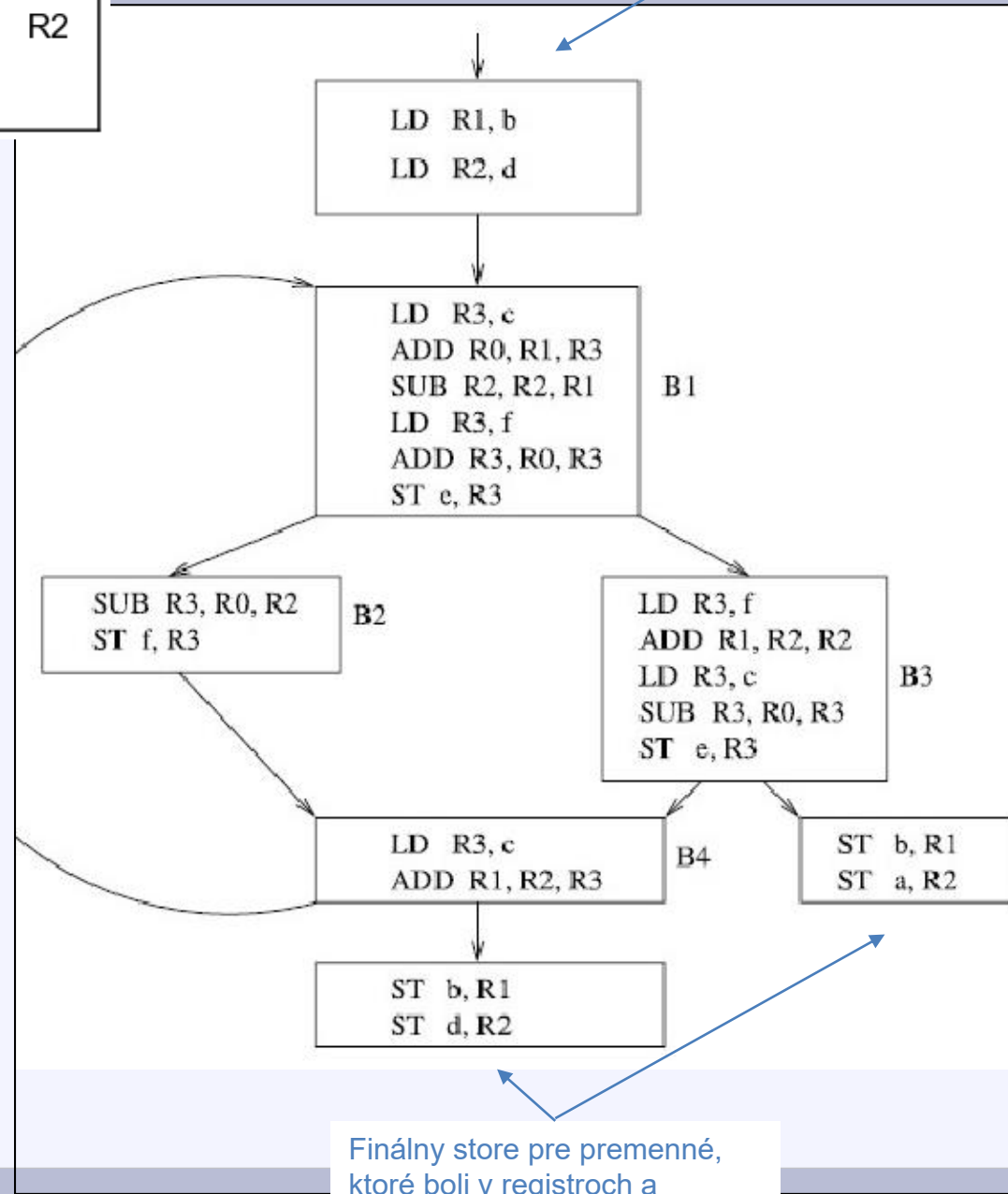
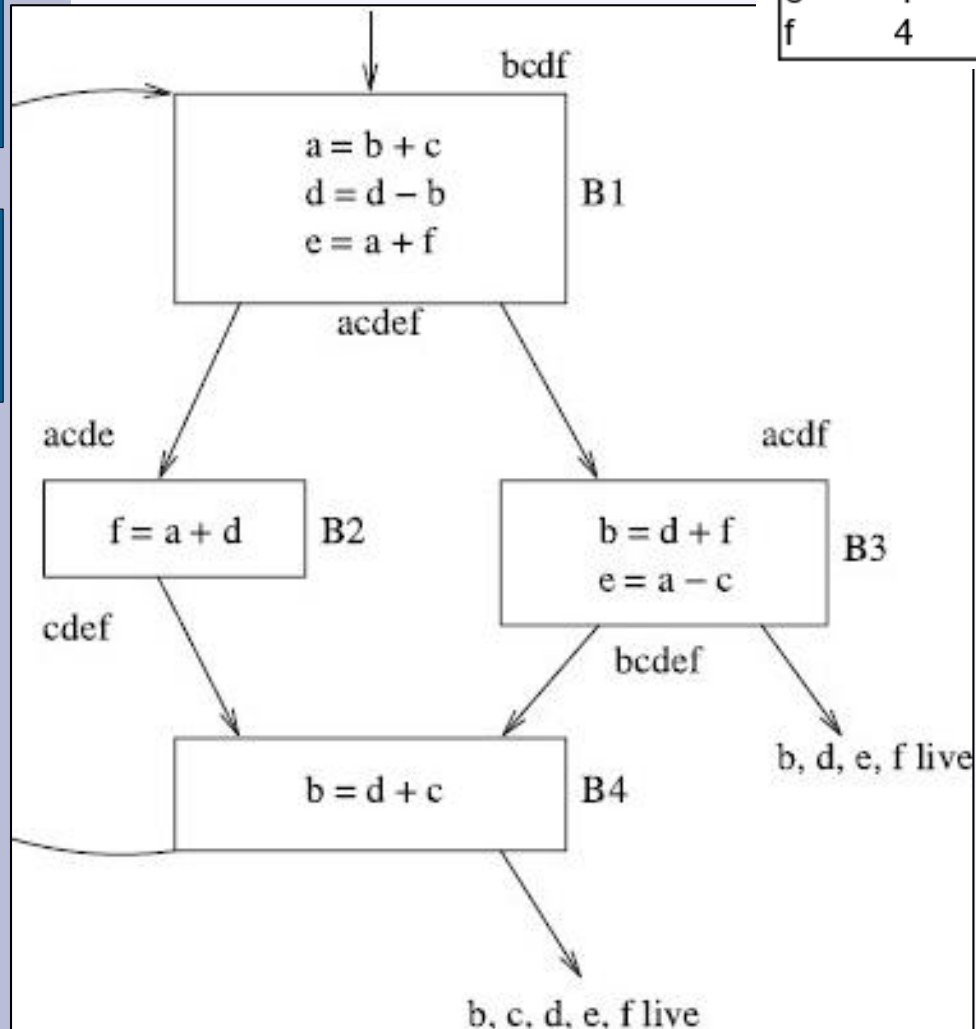
Nie je zobrazený kód
pre samotné skoky

Príklad generovaný kód



Var	Hodnota	Σ
a	4	R0
b	6	R1
c	3	R2
d	6	R2
e	4	
f	4	

Inicálny load pre premenné, ktoré budú v registroch a zároveň sú živé na vstupe



Finálny store pre premenné, ktoré boli v registroch a zároveň sú živé na výstupe

Register inference graph – RIG

- RIG je graf (inak, variable inference graph)
 - ktorého uzly sú dočasné premenné t_i .
 - Hrana medzi dvoma uzlami, ak premenné v oboch uzloch sú súčasne živé.
 - Hranu konštruujeme, keď v čase definície premennej druhá je živá.
- Vlastnosti RIG
 - Chromatické číslo RIG určuje minimálny počet registrov pre implementáciu bez operácie „spill“.
 - Reálna úloha: Daný je graf RIG a číslo k (počet registrov) zistiť, či sa RIG dá zafarbiť k farbami (vrcholové farbenie).
 - Obe úlohy sú NP-úplné.

Na úrovni procedúr

Heuristika

- Reálnu úlohu môžeme riešiť nasledovnou heuristikou
 1. V grafe G nájdí vrchol n , ktorý má menej ako k susedov.
 2. Ak takýto vrchol neexistuje, graf sa nedá zafarbiť k farbami.
 3. Ak existuje, odstráň vrchol n a hrany s ním incidentné. Dostaneš graf G' . Zafarbenie G' k farbami sa dá triviálne rozšíriť na G .
 4. $G := G'$, ak G nie je prázdny graf pokračuj bodom 1.
 5. Ak si sa dostal sem (G je prázdne), tak graf sa dá zafarbiť k farbami.
 6. Postupným pripájaním vrcholov v opačnom poradí ako boli odstraňované zafarbíš pôvodný graf.
- Ak sa graf nedá zafarbiť treba spill
 - Všeobecné pravidlo vyhýbať sa spillu v najvnútornejších cykloch (opakujú sa veľa krát – „hot paths“)

Jeršovové čísla – vhodné pre výrazy

- Strom výrazu obsahuje len uzly pre premenné
 - výsledok je otec (a poznačená operácia)
(je to taký špeciálny strom výrazu)
 - operandy sú synovia
- Jeršovové číslovanie uzlov stromu výrazu
 - Listy majú číslo 1.
 - Vnútorňý uzol, ktorý má len jedného syna, má také isté číslo ako jeho syn.
 - Vnútorňý uzol s dvoma synmi má za číslo, väčšie z čísiel jeho synov.
 - Ak obaja synovia majú rovnaké číslo, potom má číslo od tohto čísla o 1 väčšie.
- Jeršovové číslo koreňa určuje počet registrov potrebných na vyhodnotenie výrazu bez zapamätania medzivýsledkov pri zachovaní štruktúry zatvoriek.

Príklad

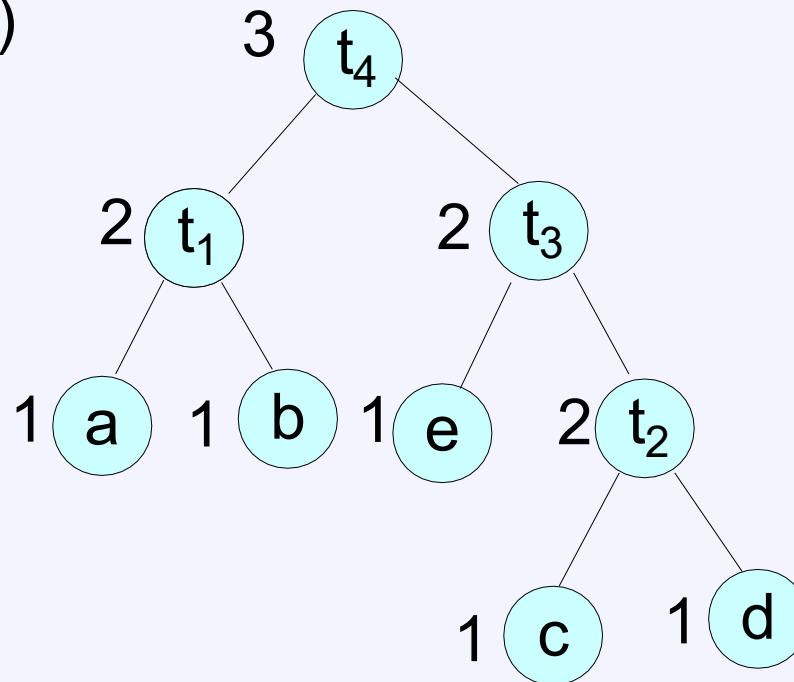
$(a - b) + e * (c + d)$

$t_1 := a - b$

$t_2 := c + d$

$t_3 := e * t_2$

$t_4 := t_1 + t_3$



```
LD  R3, d
LD  R2, c
ADD R3, R2, R3
LD  R2, e
MUL R3, R2, R3
LD  R2, b
LD  R1, a
SUB R2, R1, R2
ADD R3, R2, R3
```

Po anglicky "Ershov numbers (pomenované po A. Ershov).

„Dračí“ algoritmus generovania kódu je pre-komplikovaný.

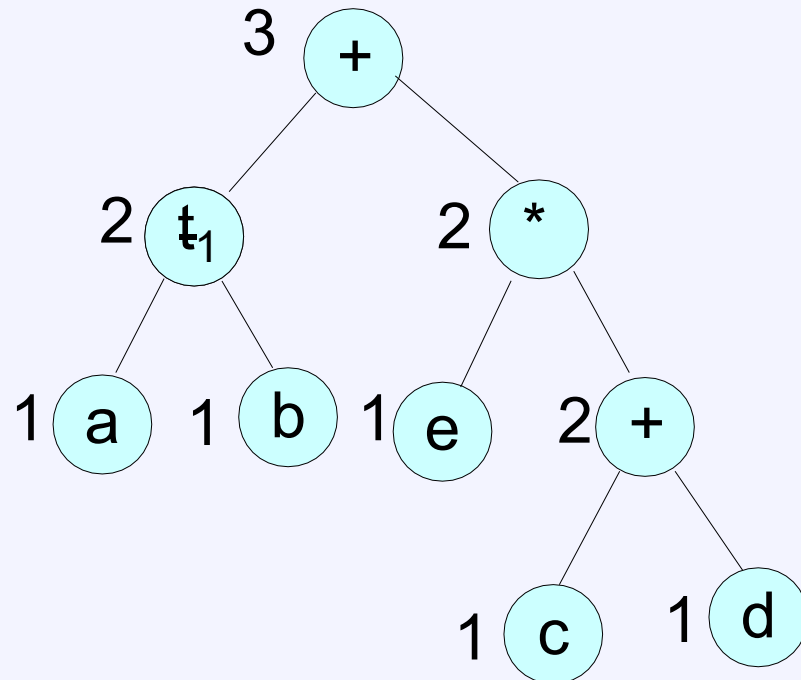
Algoritmus generovania kódu

Jeršovovými číslami riadený postorder traversing.

Ak sú čísla synov rovnaké, traverzuj v poradí ľavý syn, pravý syn.

Inak traverzuj najprv syna s väčším číslom.

Po vyhodnotení uzla si ponecháme výsledok v jednom registri.



```
LD R1, a
LD R2, b
SUB R1, R1, R2
LD R2, c
LD R3, d
ADD R2, R2, R3
LD R3, e
MUL R2, R3, R2
ADD R1, R1, R2
```

Optimalizácie cez „kľúčovú“ dierku

Cez kľúčovú dierku, peephole optimization

- Princíp
 - malé okienko, vidí len malý úsek kódu, lokálne optimalizácie **vygenerovaného strojového kódu**
- Čo optimalizujeme
 - Eliminácia nedostupného kódu (blok bezprostredne za príkazom skoku na hlavičku, do ktorého nevedie žiaden skok)
 - Optimalizácia toku riadenia
 - Skok na skok
 - Nešikovné podmienky
 - Zbytočné LD a ST inštrukcie (klasika)
 - LD a, R0
 - ST R0, a
 - Algebraické zákony (často je lepšie aplikovať ich už na medzijazyku)
 - Využitie „machine idioms“ (inc vs +1)

Príklady

SKOK CEZ SKOK:

```
if debug == 1 goto L1
    goto L2 ...
L1: printf(debug information)
L2:
```

Po optimalizácii:

```
if debug != 1 goto L2
    printf(debug information)
L2:
```

Naviac, ak je debug nastavený na 0 na začiatku programu -> constant propagation:

```
if 0 != 1 goto L2
    printf(debug information)
L2:
```

SKOK NA SKOK:

```
if a < b goto L1
    ...
L1: goto L2
```

Po optimalizácii

```
if a < b goto L2
    ...
L1: goto L2
```

Ak už nie sú skoky na L1, dá sa odstrániť aj „L1: goto L2“

..

→ Dá sa zredukovať na „goto L2“ a všetko vypisovanie debug. informácií sa stane nedostupným kódom

Výber inštrukcií

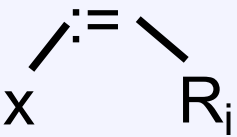
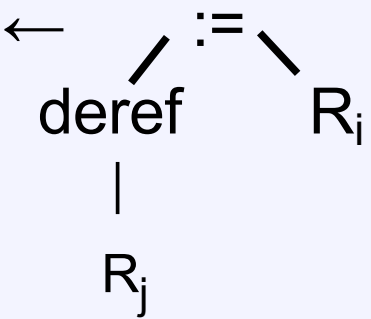
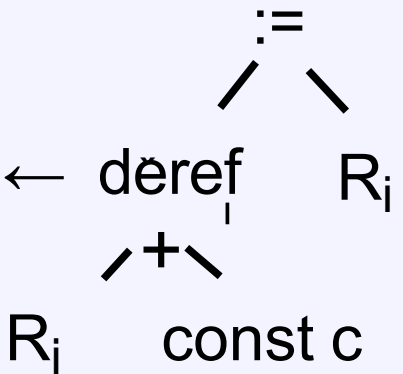
Generovanie kódu pokrývaním – tiling

- Myšlienka
 - daný je program v medzijazyku „ozdobený syntaktický strom“ alebo strom výrazu.
 - a dané sú „kachličky“ – inštrukcie resp. skupiny inštrukcií.
 - kachličky majú svoju cenu
- Skupina inštrukcií (inštrukcia) pokrýva nejaký vzor (súvislú časť) kódu v medzijazyku.
- Cieľom je (najlacnejšie) pokrytie programu v medzijazyku
- Metóda

Opakujeme nasledujúce tri kroky:

 1. nájdí vzor
 2. nahraď ho výsledkom (obvykle register obsahujúci výsledok)
 3. umiestni kachličku a zvýš cenu o cenu kachličky pokiaľ celý strom nie je pokrytý
- Stromovo-prekladová schéma

Niektoré vzory a kachličky 1

výsledok	vzor	kachlička	Štvorice
R_i	$\leftarrow \text{const } a$	$\{ \text{LD } R_i \#a \}$	$\langle :=, \text{const } a, \text{nill}, R_i \rangle$
R_i	$\leftarrow x$	$\{ \text{LD } R_i \text{ ref } x \}$	$\langle :=, x, \text{nill}, R_i \rangle$
M	$\leftarrow$ 	$\{ \text{ST } R_i \text{ ref } x \}$	$\langle :=, R_i, \text{nill}, x \rangle$
M	$\leftarrow$ 	$\{ \text{ST } R_i *R_j \}$	$\{ \langle :=, R_j, \text{nill}, t \rangle,$ $\quad \langle :=, R_i, \text{nill}, 0[t] \rangle \}$
R_i	$\leftarrow$ 	$\{ \text{ST } R_i \text{ c}(R_j) \}$ $\quad *R_j + \#c$	$\{ \langle +, R_j, \text{const } c, t \rangle,$ $\quad \langle :=, t, \text{nill}, R_i \rangle \}$

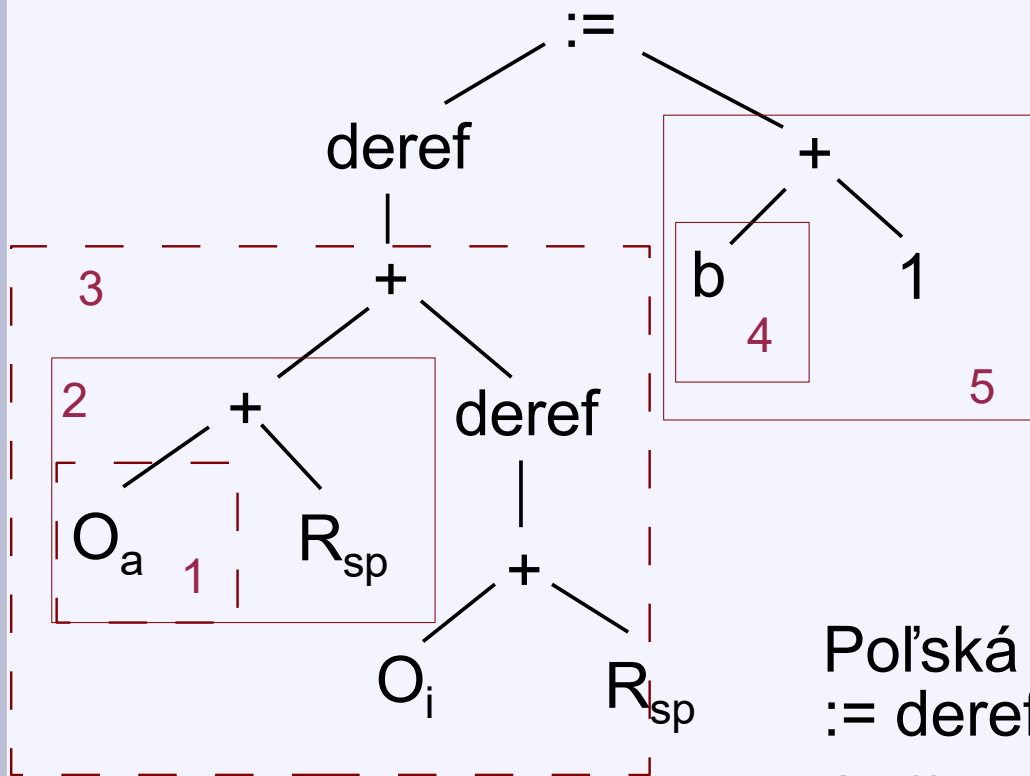
Niektoré vzory a kachličky 2

výsledok	vzor	Kachlička	štvorice
R_i	$\leftarrow \begin{array}{c} \diagup \quad + \quad \diagdown \\ R_i \quad R_j \end{array}$	$\{\text{ADD } R_i \ R_j \}$	$\langle +, R_i, R_j, R_i \rangle$
R_i	$\leftarrow \begin{array}{c} \diagup \quad + \quad \diagdown \\ R_i \quad \text{const } 1 \end{array}$	$\{\text{INC } R_i \}$	$\langle +, R_i, \text{cons } 1, R_i \rangle$
R_i	$\leftarrow \begin{array}{c} \diagup \quad + \quad \diagdown \\ R_i \quad \text{deref} \\ \\ \begin{array}{c} \diagup \quad + \quad \diagdown \\ R_j \quad \text{const } a \end{array} \end{array}$	$\{\text{ADD } R_i \ a(R_j) \}$	$\{ \langle +, R_j, \text{cons } a, R_j \rangle, \langle :=, 0[R_j], \text{nill}, t \rangle, \langle +, R_i, t, R_i \rangle \}$

Pozn.: Veľké a zložité vzory sú dôsledkom zložitých inštrukcií a adresných módov CISC architektúry.

Príklad

$a[i] := b + 1$



1. LD R_0 O_a
2. ADD R_0 R_{sp}
3. ADD R_0 ($R_{sp} + O_i$)
4. LD R_1 b
5. INC R_1
6. ST R_1 * R_0

Poľská prefixová forma:

$:= \text{deref} + + O_a R_{sp} \text{deref} + O_i R_{sp} + b 1$

Suffixová forma:

$O_a R_{sp} + O_i R_{sp} + \text{deref} + \text{deref} b 1 + :=$

„Ozdobený“ strom

Pole **a** je v call stacku:

- O_a , O_i – offsety
- R_{sp} – register so smerníkom na začiatok aktuálneho aktivačného záznamu

b je globálna premenná v pamäti

Iný príklad

Harry H. Porter:

<https://web.cecs.pdx.edu/~harry/compilers/slides/Tiling.pdf>

Jednoduchý algoritmus – maximal munch

- Najjednoduchšou technikou pre pokrytie stromu je pažravý algoritmus zhora-nadol, známy aj ako **maximal munch** (maximálne sústo)
- Pokry koreň s najväčšou možnou kachličkou (vzhľadom na počet uzlov) a potom pokračuj rekurzívne na deťoch
 - Ak existujú dve alebo viac najväčších kachličiek, vyber tú s najnižšou cenou
- Predpokladá sa, že je možné pokryť každý typ uzla (inak by niektoré kachličky boli nepoužiteľné)

Napr. predpokladajme, že vieme pokryť $OP(MEM(*), MEM(*))$, $OP(*, *)$, a $MEM(R(*))$, ale nie $R(*)$.

(Ok, toto je nepravdepodobné, ale ...).

Potom nechceme použiť najväčšiu kachličku pre koreň $OP(MEM(R(1)), MEM(R(2)))$.

Jednoduchý algoritmus – maximal munch

- Maximal munch nie nutne vyberie optimálne pokrytie, optimalizuje iba lokálne
- Ale
 - Ľahko sa implementuje
 - Väčšinou funguje veľmi dobre pre aktuálne ISA (inštrukčné sady)

Pokrývanie syntaxou riadeným prekladom

- Kachličkovanie (stromovo-prekladovú schému) nahradíme syntaxou riadeným prekladom
- Vstup: lineárny zápis medzijazyka
- Gramatika lineárny zápis pravidiel nahradzovania vzoru
- „Drak“ doporučuje prefixovú notáciu a LALR metódu syntaktickej analýzy.
 - Uprednostňuje sa ľavý operand (ak máme op $E_1 E_2$, parser sa rozhoduje na základe prefixu E_1)

Možno použiť aj suffixovú formu, ale potom sa uprednostňujú pravé operandy. Pri použití kontext senzitívnej analýzy, problém vyrieši vhodný kontext.

Príklad

- | | |
|--|-------------------------------------|
| 1) $R_i \rightarrow x$ | $\{ \text{LD } R_i \ x \}$ |
| 2) $M \rightarrow := x \ R_i$ | $\{ \text{ST } R_i \ x \}$ |
| 3) $M \rightarrow := \text{deref } R_j \ R_i$ | $\{ \text{ST } R_i \ (R_j) \}$ |
| 4) $R_i \rightarrow \text{deref} + R_j \text{ cons } a$ | $\{ \text{LD } R_i \ (R_j + a) \}$ |
| 5) $R_i \rightarrow + R_i \text{ deref} + R_j \text{ cons } a$ | $\{ \text{ADD } R_i \ (R_j + a) \}$ |
| 6) $R_i \rightarrow + R_i \ R_j$ | $\{ \text{ADD } R_i \ R_j \}$ |
| 7) $R_i \rightarrow \text{cons } a$ | $\{ \text{LD } R_i \ \#a \}$ |
| 8) $R_i \rightarrow + R_i \text{ cons } 1$ | $\{ \text{INC } R_i \}$ |
| 9) $R_{sp} \rightarrow sp$ | |
| 10) $M \rightarrow x$ | |

- Gramatika je nejednoznačná, riešenie konfliktov:

Preferujeme väčší reduce pred menším, shift pred reduce.

Optimálny algoritmus

- Algoritmus zhora-nadol

Pre každú kachličku, ktorá pokrýva koreň

vypočítaj optimálne pokrytie každého podstromu danej kachličky

vypočítaj cenu použitia daného pokrytia a aktuálnej kachličky

Vyber minimum z týchto pokrytí (Generuje kód v opačnom poradí.)

- Funguje tento algoritmus? Áno. Skúša všetky kachličky, ktoré pokrývajú koreň. Toto zabezpečí, že vyberie tú, ktorej použitie vedie k najmenšej cene pre celý strom.
- Je algoritmus efektívny? Nie. Robí opakovanú prácu a môže byť exponenciálny.
- Môžeme to urobiť efektívnejšie? Áno,
- **Dynamické programovanie.**

Optimálny algoritmus – dynamické programovanie

- Algoritmus využívajúci dynamické programovanie postupuje v nasledujúcich fázach:
 1. Pre každý uzol **N** ozdobeného parsovacieho stromu **T** vypočítaj zdola-nahor pole **C** možných pokrytí podstromu v **N** a ich cien ($C = \text{cost vector}$). V prípade výrazov, **C[i]** je cena minimálneho pokrytia, ak použijeme **i** registrov ($1 \leq i \leq r$, kde **r** je celkový počet registrov).
 - Pre každú hodnotu **C[i]** sa pamätá aj informácia, ktorá kachlička sa použila a koľko registrov bolo priradených jednotlivým deťom resp. či sa „spill-ovali“ do pamäte
 - V **C[0]** je cena minimálneho pokrytia podstromu v **N**, ak sa výsledok výrazu pod **N** uloží do pamäte („spill“)
 2. Pretraverzuj strom **T** a za použitia cenových vektorov urči, ktoré postromy budú mať výsledok uložený v pamäti
 3. Pretraverzuj podstromy, ktorých výsledok bude v pamäti, a prislúchajúce inštrukcie, a za použitia cenových vektoroch pre ne vygeneruj cieľový kód
 4. Urob to isté ako v kroku 3 pre celý strom **T** a vygeneruj preň cieľový kód

Zložitosť

- Každá fáza sa dá realizovať v lineárnom čase.
- Optimálne pokrytie stromu je teda lineárny problém vzhľadom na veľkosť stromu
- Neeliminuje spoločné podvýrazy
- Algoritmus pokrývania a dynamického programovania sa dá realizovať aj na dagu. Zložitosť môže byť rovnaká ako na strome, ak cena zapamätania spoločnej časti je väčšia ako jej opätovný výpočet, alebo ak v optimálnom programe treba spoločnú časť počítat dvomi rôznymi spôsobmi.
- Strom môže byť až exponenciálne väčší ako dag.
- V praxi rozdiely nie sú až také dramatické.